Machine Learning With Python Project

Global E-Commerce RFM Segmentation & Customer Value Prediction

Data Connect

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv(r"C:\Users\Sathish_P\Downloads\Data Analytics Course\End - To -End Project\Global E-commerce Mlr Projec
df
```

	CustomerID	Country	Age	Gender	MembershipTier	RecencyDays	Frequency	AvgOrderValue	MonetaryValue	HighValueCustomer
0	EMP-10001	Germany	43	Male	Standard	4	7	186.96	1186.11	1
1	EMP-10002	United States	22	Male	Standard	33	6	78.89	518.56	0
2	EMP-10003	United States	41	Other	Platinum	68	6	75.52	483.07	0
3	EMP-10004	United Kingdom	56	Female	Gold	24	4	60.81	248.16	0
4	EMP-10005	Germany	20	Male	Standard	15	6	159.71	1045.28	1
...	...	...	...	...	...	...	...	...	...	...
2995	EMP-12996	Germany	64	Male	Standard	17	5	90.62	448.49	0
2996	EMP-12997	Germany	61	Male	Standard	94	5	134.14	627.54	0
2997	EMP-12998	United States	42	Male	Silver	57	6	178.93	1120.58	1
2998	EMP-12999	United Kingdom	26	Male	Silver	27	4	84.30	349.79	0
2999	EMP-13000	United Kingdom	18	Female	Silver	77	5	79.86	423.66	0

3000 rows × 10 columns

Data Cleaning

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   CustomerID         3000 non-null   object 
 1   Country            3000 non-null   object 
 2   Age                3000 non-null   int64  
 3   Gender             3000 non-null   object 
 4   MembershipTier     3000 non-null   object 
 5   RecencyDays        3000 non-null   int64  
 6   Frequency          3000 non-null   int64  
 7   AvgOrderValue      3000 non-null   float64
 8   MonetaryValue      3000 non-null   float64
 9   HighValueCustomer  3000 non-null   int64  
dtypes: float64(2), int64(4), object(4)
memory usage: 234.5+ KB
```

```
df.head()
```

	CustomerID	Country	Age	Gender	MembershipTier	RecencyDays	Frequency	AvgOrderValue	MonetaryValue	HighValueCustomer
0	EMP-10001	Germany	43	Male	Standard	4	7	186.96	1186.11	1
1	EMP-10002	United States	22	Male	Standard	33	6	78.89	518.56	0
2	EMP-10003	United States	41	Other	Platinum	68	6	75.52	483.07	0
3	EMP-10004	United Kingdom	56	Female	Gold	24	4	60.81	248.16	0
4	EMP-10005	Germany	20	Male	Standard	15	6	159.71	1045.28	1

```
X = df.drop(columns=['CustomerID', 'MonetaryValue', 'HighValueCustomer', 'Cluster'])
y = df['HighValueCustomer']
```

```
X
```

	Country	Age	Gender	MembershipTier	RecencyDays	Frequency	AvgOrderValue
0	Germany	43	Male	Standard	4	7	186.96
1	United States	22	Male	Standard	33	6	78.89
2	United States	41	Other	Platinum	68	6	75.52
3	United Kingdom	56	Female	Gold	24	4	60.81
4	Germany	20	Male	Standard	15	6	159.71
...	...	...	...	...	...	...	...
2995	Germany	64	Male	Standard	17	5	90.62
2996	Germany	61	Male	Standard	94	5	134.14
2997	United States	42	Male	Silver	57	6	178.93
2998	United Kingdom	26	Male	Silver	27	4	84.30
2999	United Kingdom	18	Female	Silver	77	5	79.86

3000 rows × 7 columns

```
y
```

```
0       1
1       0
2       0
3       0
4       1
       ..
2995    0
2996    0
2997    1
2998    0
2999    0
Name: HighValueCustomer, Length: 3000, dtype: int64
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import numpy as np
```

```
# Log transform to normalize skewed financial data
rfm_log = np.log1p(df[['RecencyDays', 'Frequency', 'MonetaryValue']])
rfm_log
```

	RecencyDays	Frequency	MonetaryValue
0	1.609438	2.079442	7.079277
1	3.526361	1.945910	6.252982
2	4.234107	1.945910	6.182230
3	3.218876	1.609438	5.518095
4	2.772589	1.945910	6.952996
...	...	...	...
2995	2.890372	1.791759	6.108114
2996	4.553877	1.791759	6.443400
2997	4.060443	1.945910	7.022494
2998	3.332205	1.609438	5.860188
2999	4.356709	1.791759	6.051289

3000 rows × 3 columns

```
scaler = StandardScaler()
scaler
```

StandardScaler

▶ Parameters

```
rfm_scaled = scaler.fit_transform(rfm_log)
rfm_scaled
```

```
array([[-1.59828354,  0.53334318,  1.14509586],
       [ 0.17029789,  0.13444591,  0.05768678],
       [ 0.82327489,  0.13444591, -0.03542431],
       ...,
       [ 0.66305036,  0.13444591,  1.07036859],
       [-0.00883335, -0.87069493, -0.4592333 ],
       [ 0.93638963, -0.32604719, -0.20774306]], shape=(3000, 3))
```

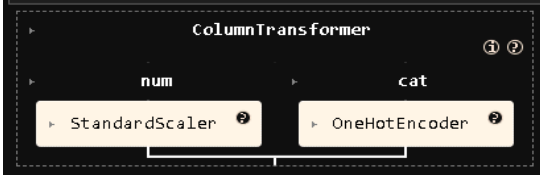
```
# K-Means Clustering
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(rfm_scaled)
df['Cluster']
```

```
0       0
1       2
2       2
3       1
4       0
       ..
2995    0
2996    2
2997    3
2998    1
2999    2
Name: Cluster, Length: 3000, dtype: int32
```

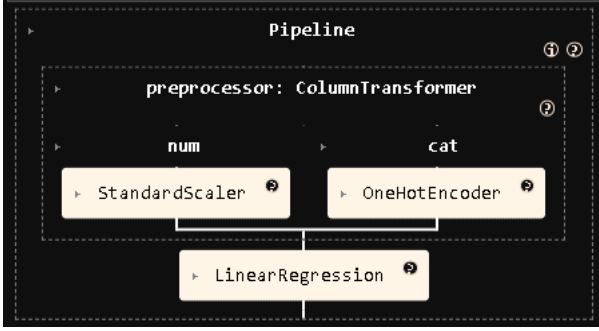
```python
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```python
# 3. Train-Test Split (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```python
# 4. Preprocessing Pipeline (Scaling numericals, One-Hot Encoding categoricals)
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), categorical_features)
    ]
)
preprocessor
```




```python
# Combine Preprocessor and Linear Regression into a Pipeline
mlr_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])
mlr_pipeline
```




```python
# 5. Train and Predict
mlr_pipeline.fit(X_train, y_train)
y_pred = mlr_pipeline.predict(X_test)

# Print Performance Metrics
print(f"--- Multiple Linear Regression Performance ---")
print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"Root Mean Squared Error (RMSE): ${np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")
```

```
--- Multiple Linear Regression Performance ---
R2 Score: 0.5732
Root Mean Squared Error (RMSE): $0.28
```

```python
# 6. Professional Visualizations for Interviews
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
sns.set_theme(style="whitegrid")

# Graph 1: Actual vs. Predicted Values Scatter Plot
axes[0].scatter(y_test, y_pred, alpha=0.6, color='royalblue', edgecolors='k')
axes[0].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2, label='Ideal Fit (y = x)')
axes[0].set_xlabel('Actual Monetary Value ($)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Predicted Monetary Value ($)', fontsize=12, fontweight='bold')
axes[0].set_title('MLR: Actual vs. Predicted Customer Spend', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=11)

# Graph 2: Residuals Plot (Checking Homoscedasticity)
residuals = y_test - y_pred
axes[1].scatter(y_pred, residuals, alpha=0.6, color='darkorange', edgecolors='k')
axes[1].axhline(y=0, color='r', linestyle='--', lw=2, label='Zero Error Line')
axes[1].set_xlabel('Predicted Values ($)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Residuals (Actual - Predicted)', fontsize=12, fontweight='bold')
axes[1].set_title('MLR: Residuals Plot (Error Variance)', fontsize=14, fontweight='bold')
axes[1].legend(fontsize=11)

plt.tight_layout()
plt.show()
```



